GOOGLE PLAY STORE

ANALYTICS

Data Analytics, Statistical Testing,
Sentiment Analysis & Predictive Modelling

PROJECT REPORT

Prepared by

Yaseen Akthar Maniyar

B.Tech – Artificial Intelligence & Robotics
Dayananda Sagar University
Bengaluru, Karnataka, India

Report integrity note: This report documents the project workflow and methodology based on the available project details. Actual dataset-specific values (p-values, model scores, sentiment percentages, category rankings and sample counts) are not available here and have not been invented. Fill the marked result fields using the output from your executed notebook before submission.

Abstract

The Google Play Store Analytics project examines application metadata and user-review information to understand patterns in ratings, review activity, installations, pricing, categories and user sentiment. The workflow combines data cleaning, feature engineering, exploratory data analysis (EDA), sentiment classification, a custom Success Score, statistical hypothesis testing, supervised machine learning and an interactive HTML dashboard. A Welch independent two-sample t-test is used to investigate whether average ratings differ between free and paid apps. A Random Forest classifier predicts a categorical Rating_Group from selected app characteristics while excluding the original Rating field to reduce direct target leakage. The dashboard provides filters for category, app type, content rating, year, minimum installs and rating range. This report describes the approach, interpretation framework and limitations. Empirical findings must be completed from the executed notebook.

Keywords

Google Play Store, data analytics, exploratory data analysis, sentiment analysis, Welch's t-test, Random Forest, feature importance, business intelligence.

Executive Summary

This project is designed as an end-to-end analytics workflow rather than a collection of isolated charts. Application metadata is cleaned and transformed; review information is used for sentiment analysis where available; derived features support comparisons and ranking; statistical testing evaluates a defined hypothesis; and a classification pipeline predicts rating groups. An interactive HTML dashboard makes the analysis accessible to viewers. Numerical findings should be copied from the notebook after execution.

Component	Purpose	Output
Data preparation	Improve consistency and field types	Clean application/review data
EDA	Describe distributions and compare categories	Charts and descriptive summaries
Sentiment analysis	Summarize review polarity	Positive/Neutral/Negative classifications when scores exist
Success Score	Create a custom app ranking metric	Rating × log(1 + Reviews)
Statistical test	Compare average ratings of paid and free apps	t-statistic, p-value, confidence interval, effect size
Predictive analytics	Predict Rating_Group	Random Forest metrics and confusion matrix
Dashboard	Enable interactive exploration	Standalone HTML dashboard

1. Introduction

Mobile applications differ in categories, user ratings, review volumes, reported installs, pricing models, content ratings and update histories. Analyzing these fields together can describe the application ecosystem and identify associations potentially useful to developers, analysts and product teams. This project uses application metadata and user-review information to demonstrate a reproducible analytics workflow.

The analysis is observational. Associations should not be interpreted as proof of causation. Findings may also reflect the age, coverage and collection method of the source datasets.

2. Problem Statement

How can Google Play Store application metadata and user reviews be analyzed to identify category-level performance patterns, summarize user sentiment, test differences between paid and free applications, and estimate an application's rating group using machine-learning methods?

3. Objectives

- Clean application and review data and prepare analysis-ready fields.
- Explore distributions and relationships among ratings, reviews, installs, prices, categories, sizes and update years.
- Analyze user-review sentiment where review text or sentiment scores are available.
- Create and examine a custom Success Score for ranking apps.
- Test whether mean ratings differ between paid and free apps using Welch's t-test.
- Train and evaluate a Random Forest classifier for Rating_Group without using Rating as an input.
- Present indicators and visualizations in an interactive HTML dashboard.
- Document limitations and recommend future improvements.

4. Dataset Description

The notebook uses two dataframes, apps_df and reviews_df. Application metadata includes App, Category, Rating, Reviews, Size, Installs, Type, Price, Content Rating, Genres, Last Updated, Current Ver and Android Ver. The review table contains app-associated review information and may include translated review text and sentiment-related columns.

Field	Purpose	Preparation notes
App	Application identifier/name	Check missing names and duplicates
Category / Genres	Application classification	Categorical features
Rating	Store rating on a 0–5 scale	Numeric; validate range
Reviews	Review count	Numeric; typically skewed
Installs	Reported install count/range	Remove formatting characters during parsing
Size	Reported app size	Convert to a consistent unit such as MB
Type / Price	Free/paid status and listed price	Normalize labels and numeric price
Content Rating	Audience/content classification	Categorical feature
Last Updated / Year	Update date and derived year	Parse dates; invalid dates become missing
Review text / Sentiment_Score	Review polarity analysis	Handle missing or unusable text

Record the exact dataset source, original row counts, date coverage and final row counts after cleaning from your notebook. These values are not supplied in the current project context and are not guessed.

5. Tools and Technologies

Tool	Role
Python	Main programming language

Pandas / NumPy	Data cleaning, transformations, grouping and numerical calculations
Matplotlib / Seaborn	Exploratory and statistical visualizations
Plotly	Interactive charts and HTML dashboard
SciPy	Welch t-test and statistical calculations
Scikit-learn	Preprocessing, Random Forest and evaluation metrics
NLTK / VADER	Lexicon-based sentiment scoring where used
Google Colab / Jupyter	Notebook execution environment

6. Data Cleaning and Preprocessing

1. Load the application and review CSV files into dataframes.
2. Inspect column names, data types, missing values and duplicate records.
3. Remove duplicate application rows where appropriate and record the number removed.
4. Validate ratings and exclude invalid values outside the expected 0–5 range.
5. Convert Reviews and Installs to numeric values after removing formatting characters.
6. Convert Price to numeric after removing currency symbols.
7. Normalize Size into a consistent unit; retain missing values where source information is unavailable.
8. Parse Last Updated as a datetime and derive Year.
9. Handle missing review text and do not automatically treat missing sentiment as neutral.
10. Verify final dataframe dimensions and transformed data types.

7. Feature Engineering

7.1 Log-transformed variables

Review and install counts are often right-skewed. The project creates Log_Reviews = log(1 + Reviews) and Log_Installs = log(1 + Installs). Log transformations make highly skewed counts easier to inspect but do not guarantee normally distributed data.

7.2 Rating_Group

The current mapping is: Rating ≥ 4 → Top rated; 3 ≤ Rating < 4 → Above average; 2 ≤ Rating < 3 → Average; Rating < 2 → Below average. Rows with missing ratings should remain missing or be excluded from target modelling.

7.3 Estimated revenue proxy

The project defines Revenue = Price × Installs. This is an estimated revenue proxy, not verified revenue. Install values may be ranges, and the calculation does not account for platform fees, taxes, refunds, subscriptions, in-app purchases, regional prices or actual paid conversions. Label charts and conclusions as “estimated revenue proxy.”

7.4 Success Score

$$\text{Success_Score} = \text{Rating} \times \log(1 + \text{Reviews})$$

This custom score combines rating and review volume. It is not an official Google Play metric. Since Rating and Reviews are built into the formula, relationships between the score and either input are partly mathematical by construction and should not be presented as independent evidence of causality or overall business success.

8. Exploratory Data Analysis

EDA uses summary statistics, grouped tables and visualizations to inspect the app dataset. Conclusions should refer to the actual plots and tables produced by the notebook.

Analysis	Questions	Suggested output
Category distribution	Which categories contain the most apps?	Count plot and top-category table
App type	What share of apps are free versus paid?	Count and percentage table
Ratings	How are ratings distributed?	Histogram and descriptive statistics
Reviews and installs	How concentrated are engagement counts?	Log-scale histograms/scatter plots
Category installs	Which categories have the largest reported install totals?	Ranked bar chart
Revenue proxy	Which categories have the largest Price × Installs proxy?	Ranked bar chart labelled estimated
Update year	How are apps distributed by update year?	Yearly count chart
Paid vs free	How do rating distributions compare?	Box plot and hypothesis test
Rating vs reviews	Is review volume associated with rating?	Scatter plot with log-scaled x-axis if appropriate

Distinguish total installs from per-app averages or medians. A category with more listed apps may have a larger total simply because it contains more applications.

9. Sentiment Analysis

The review-analysis workflow uses sentiment information to summarize expressed polarity. With VADER, compound scores typically range from −1 to +1. The proposed thresholds classify scores ≥ 0.05 as Positive, scores ≤ −0.05 as Negative, and scores between those thresholds as Neutral.

Class	Threshold	Interpretation
Positive	Compound ≥ 0.05	Positive polarity under the selected lexicon
Neutral	−0.05 < compound < 0.05	Weak or balanced polarity
Negative	Compound ≤ −0.05	Negative polarity under the selected lexicon

Sentiment is a text-based proxy, not a definitive measure of customer satisfaction. Sarcasm, multilingual text, short reviews and domain-specific language can affect performance. State how app-level sentiment is aggregated, and calculate percentages only over reviews successfully classified.

10. Statistical Analysis: Paid vs Free Ratings

10.1 Research question

Is there a statistically significant difference in average ratings between paid and free applications?

10.2 Hypotheses

H_0: The population mean rating is equal for paid and free applications.
H_1: The population mean rating differs between paid and free applications.

10.3 Method

Welch's independent two-sample t-test is used because it does not assume equal population variances. The significance level is $\alpha = 0.05$. Report the valid sample sizes, group means and standard deviations, t-statistic, p-value, 95% confidence interval for the mean difference and an effect-size estimate such as Cohen's d.

10.4 Decision and interpretation

- If $p < 0.05$, reject H_0 and report evidence of a statistically significant difference in average ratings.
- If $p \geq 0.05$, fail to reject H_0; this does not prove that the group means are identical.
- Interpret the confidence interval and effect size alongside the p-value. Statistical significance does not necessarily imply a large practical difference.

RESULTS TO INSERT FROM NOTEBOOK: Free mean rating = [value]; paid mean rating = [value]; t-statistic = [value]; p-value = [value]; 95% CI = [lower, upper]; Cohen's d = [value]; decision = [reject / fail to reject H_0].

Assumptions and caveats: observations should be reasonably independent; outliers and the bounded 0–5 rating scale should be considered. This observational comparison cannot establish that charging a price causes a rating change.

11. Predictive Analytics: Random Forest Classifier

11.1 Target and features

The target is Rating_Group, derived from Rating. Planned input features are Category, Reviews, Size, Installs, Price, Type, Content Rating, Genres and Year, subject to availability in the cleaned dataframe.

11.2 Leakage prevention

Do not include Rating or a direct derivative of Rating as an input because Rating_Group is calculated from Rating. This prevents directly giving the model its target. Reviews and installs may be contemporaneous with the rating, so this is a retrospective classification experiment—not a guaranteed forecast of future app performance.

11.3 Modelling pipeline

11. Separate input features X from target y.
12. Split data into training and test sets, using a fixed seed and stratification when class counts permit.
13. Impute missing numerical values using a median and categorical values using the most frequent value.
14. One-hot encode categorical fields and ignore unseen categories at inference time.
15. Train a Random Forest classifier on the training set.
16. Predict rating groups for the held-out test set.
17. Report metrics and inspect errors using the confusion matrix.

11.4 Evaluation metrics

Metric	Meaning	Caution
Accuracy	Share of predictions that are correct	Can mislead with imbalanced classes

Precision	Share of predicted class cases that are correct	Inspect per-class values too
Recall	Share of actual class cases correctly recovered	Low recall indicates missed cases
F1-score	Harmonic mean of precision and recall	State the averaging method
Confusion matrix	Counts correct and incorrect class assignments	Shows commonly confused groups
Feature importance	Relative usefulness for the fitted forest's splits	Not causal; can favour high-cardinality variables

MODEL RESULTS TO INSERT: Test accuracy = [value]; weighted precision = [value]; weighted recall = [value]; weighted F1-score = [value]. Add the classification report, confusion matrix and top 10 features from the executed notebook. No model metrics have been invented.

Compare the model with a majority-class baseline. A single train/test split can be unstable; cross-validation and permutation importance can strengthen the analysis. Feature importance describes model-specific predictive usefulness, not proof that a feature causes higher ratings.

12. Interactive Business Intelligence Dashboard

The standalone HTML dashboard uses Plotly charts and browser-side JavaScript filters to help users explore the data.

Component	Purpose
KPI cards	App count, average rating, total reported installs and estimated revenue proxy
Category filter	Limit the view to a selected category
App type filter	Compare free and paid applications
Content rating filter	Focus on a selected audience classification
Year filter	Filter by update year where available
Minimum installs filter	Focus on applications above an install threshold
Rating range	Restrict the view to a rating interval
Category charts	Compare installs and estimated revenue proxy by category
Rating charts	Inspect rating distribution and reviews versus rating
Sentiment and rating groups	Summarize labels and rating categories
Top apps table	Rank apps by the custom Success Score
Feature importance	Display fitted-model importance when model output is available

Dashboard path: dashboard/Google_Play_Store_Interactive_Dashboard.html. Test each filter combination, empty-result behaviour, missing sentiment, zero values on log axes and category labels. Ensure the feature-importance chart contains actual fitted-model results before presenting it as an ML finding. The provided HTML implementation references Plotly through a CDN, so internet access may be required to load the chart library.

13. Business Interpretation Framework

- Category performance: distinguish total installs from average installs per app and consider category size.
- Estimated revenue proxy: report category rankings but state the proxy's limitations.
- Ratings and engagement: describe observed associations without causal claims.

- Paid versus free: report the Welch test, confidence interval and effect size together.
- Sentiment: state class percentages and the denominator of classified reviews.
- Success Score: explain that the formula rewards rating and review volume by design.
- Prediction: compare metrics with a baseline and describe common confusion-matrix errors.
- Feature importance: identify top predictors from the fitted model but do not interpret importance causally.

14. Limitations

- The dataset may be historical and may not represent the current Play Store.
- Install values may be ranges; revenue estimates based on them are approximate.
- Listed price multiplied by installs does not equal actual developer revenue.
- Ratings and review counts may be skewed and affected by selection bias.
- Lexicon sentiment methods can misread sarcasm, context, multilingual text or domain-specific language.
- Rating_Group depends on selected thresholds and is derived from Rating.
- Feature importance does not establish causation.
- A single train/test split can produce variable metrics; cross-validation and external validation would be stronger.
- Actual printed metrics and plots were not available when this report was generated; insert them before submission.

15. Future Scope

- Use cross-validation and compare Random Forest with Logistic Regression, Decision Tree and boosting methods.
- Consider rating regression with careful leakage control and a defined prediction time.
- Use permutation importance or SHAP-style explanations for interpretability.
- Improve review analysis with language detection and domain-aware NLP.
- Add confidence intervals and robust effect sizes to other comparisons.
- Deploy the dashboard as a hosted static page or lightweight web application.
- Automate data validation, insight generation and report creation.
- Explore app recommendation or category opportunity analysis with an explicit business objective.

16. Conclusion

This project establishes an end-to-end framework for Google Play Store analytics. It combines data preparation and EDA with review sentiment, a custom Success Score, a hypothesis test comparing paid and free ratings, Random Forest classification and an interactive HTML dashboard. The workflow demonstrates how descriptive, inferential, predictive and business-intelligence methods can be integrated in a portfolio project. Final conclusions about category leaders, statistical significance, sentiment balance and model performance must be based on executed notebook outputs; this report intentionally avoids numerical claims that were not available.

17. Reproducibility and Repository Structure

```
Google-Play-Store-Analytics/
├── README.md
```

```
├── LICENSE
├── requirements.txt
├── .gitignore
├── data/README.md
├── notebooks/Google_Play_Store_Analytics.ipynb
├── dashboard/Google_Play_Store_Interactive_Dashboard.html
├── results/figures/
├── results/final_insights.txt
└── report/Google_Play_Store_Analytics_Report.pdf
```

Suggested requirements: pandas, numpy, matplotlib, seaborn, plotly, scipy, scikit-learn, nltk, kaleido and jupyter. Capture exact package versions for reproducibility. Do not upload datasets unless their license terms permit redistribution; otherwise provide verified source and retrieval instructions.

18.Notebook

https://colab.research.google.com/drive/16x-uRub2r8yLO5oKcUO8Ftu8nVrI3G7V?usp=sharing

References

Add the verified source URL and citation for each dataset used in the notebook.

McKinney, W. (2022). Python for Data Analysis (3rd ed.). O'Reilly Media.

Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Virtanen, P., et al. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. Nature Methods, 17, 261–272.

Hutto, C. J., & Gilbert, E. (2014). VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. Proceedings of the International AAAI Conference on Web and Social Media.

Plotly Python Graphing Library documentation: https://plotly.com/python/

Scikit-learn documentation: https://scikit-learn.org/stable/